

We reduce the complexity of most neighbor pruning in insert from $O(R^2)$ to $O(R)$. This is inspired by an observation that the original neighbors mostly follow the triangular inequality. Therefore, instead of checking all the neighbor pairs, we only check the newly inserted neighbor with all the other neighbors. The farthest neighbor that does not follow triangular inequality is pruned.

If no neighbor can be pruned, we fall back to the original $O(R^2)$ pruning algorithm. We prune the first neighbor that does not follow triangular inequality, for early stopping while not breaking the characteristics of the original graph.

3.4 Buffered Delete and Merge

In §3.2 and §3.3, we show how ODINANN supports efficient direct inserts. In this section, we discuss how ODINANN supports buffered deletes and merging at a low cost. We choose buffered deletes instead of direct ones because they show little interference with frontend search during the merge (§4.7).

3.4.1 Buffered Delete

Overview. ODINANN records the IDs of deleted vectors in memory. When deleting a vector, ODINANN adds its ID to an in-memory deletion set, resulting in a low runtime memory footprint (4B or 8B per vector). Deleted vectors are excluded from the search results after the on-disk ANNS.

For concurrency control with search and insert, a global read-write lock is used. Delete operations acquire the write lock to update the delete set. After a search finishes on the disk, it acquires the read lock and excludes the deleted records from the search result. We do not use more fine-grained locks as delete has much less overhead than search and insert.

Challenge: search quality insurance. In workloads containing deletes, the on-disk search quality may be degraded by deleted vectors. Specifically, the results returned by the on-disk ANNS may contain deleted vectors, which are excluded from the results after the on-disk ANNS finishes. This process may lead to fewer than expected top- k results, even if we return more than k results in the on-disk search. In ODINANN, we should solve this issue without the assistance of the in-memory index, because it is avoided by direct insert.

Solution: dynamic candidate pool. As deleted nodes are excluded from the search result, there may be insufficient results even with a large candidate pool length l . To tackle the problem, we adapt the candidate pool size dynamically.

We find that the deleted vectors should only serve as indexing, but not contribute to the results. Motivated by this, we "ignore" the existence of deleted vectors in the candidate pool, namely, we let the candidate pool contain at most l vectors that are *not deleted*. To achieve it, we adjust the candidate pool size l after admitting the neighbors of the current exploring vector (line 12 in Algorithm 1).

3.4.2 Two-Pass Merge for Delete

When to merge. We use two metrics to decide when to merge. The first is the ratio of deleted vectors in the index. When it reaches a threshold (e.g., 10%), merging is triggered. This metric is straightforward but sometimes inaccurate. This is because vectors located in different places are not equally important. For example, near-root vectors influence every search request, while near-boundary vectors may only influence a small part. However, they are treated equally in the metric of deleted vector ratio.

We use the search I/O amplification as another metric. It is based on our observation that the disk I/O in the search process reflects the index quality. Given the same candidate pool size l , worse index quality results in a larger candidate pool, thus resulting in more disk I/O. Therefore, we record the average disk I/O during the search. Merging is triggered when the disk I/O is significantly more (e.g., 1.1×) than when no vectors are deleted.

How to merge. Merging delete in ODINANN includes two passes, in each pass ODINANN scans the whole index sequentially. This process is not as I/O intensive as merging inserts, so it shows little interference with frontend search (§4.7).

In the first pass, ODINANN scans the whole index to load the neighbor IDs of the deleted vectors into memory. Loading all of them can cause a high memory consumption (e.g., 512B for a vector with 128 neighbors). ODINANN uses a simple approach, namely loading part of the neighbors for each vector, to meet the memory budget at the cost of index quality.

In the second pass, ODINANN scans the whole index to do three things in a streaming manner. (1) Read a batch of records and replace the deleted vector IDs with their neighbor IDs in memory. (2) Prune neighbors for records whose out-degree is larger than the limit. (3) Write them back to disk.

Updates during merge. Deletes can be trivially handled in memory, so we mainly discuss inserts here. We propose two ways for inserts during merge. First, they can be absorbed by an in-memory index and digested using direct insert after the merge. Second, they can be executed with direct insert by not allocating records in the merging pages. As merge is typically short in ODINANN (§4.7), the overheads of the in-memory index and extra disk space are acceptable.

3.5 Discussion

Consistency. ODINANN uses snapshots and journaling for consistency. Merging creates an index snapshot. Between two merging, journaling is used to record the delta updates for the in-memory and on-disk data structures. For less overhead, journaling may be flushed to disk later than on-disk inserts. To maintain a consistent prefix for updates [28], we roll back the index by (buffered) deleting all the records with newer IDs than the journal. We store the metadata for recovery (e.g., ID and transaction ID) together with each record on disk.

GC-free. ODINANN supports *GC-free inserts*. Compared to the log-structured layout, ODINANN directly reuses the invalid records, eliminating the need for *logical* garbage collection – a process to gather valid records into a new index. Our designs target FTL-based SSDs and do not interfere with their *physical* garbage collection. For *deletes*, to recycle the space of deleted vectors, ODINANN needs the lightweight merge (§3.4.2), which is similar to a logical GC.

Insert latency. Compared to buffered insert in DiskANN, ODINANN has a higher insert latency as it directly writes the updated records to disk. This leads to a ~11.1ms insert latency (§4.4). We believe this latency is acceptable, as real-world systems [7, 22] target indexing latencies around 10ms.

Memory usage. ODINANN stores three things in memory:

- *PQ table* for PQ-compressed vectors, 32 bytes per vector.
- *ID-to-location and ID-to-tag hash tables* contain the mapping of vector ID to 4-byte record location and 4-byte tag (the same as DiskANN [23]), 16 bytes per vector.
- *Location-to-ID hash table* contains the mapping of record location to vector ID, used by the first pass of a merge. We use per-page fixed-size arrays for this, requiring 4 bytes per record (including empty ones) and 4 bytes per page.

For a typical ODINANN index with 2× disk space consumption and 6 records per page, the memory usage is ~58 bytes per vector or 58GB for billion-scale datasets. The memory usage in our evaluation is higher (83.8GB, §4.3). This is due to the memory overheads of hash tables, and free slots caused by dynamic doubling of the PQ table and hash tables.

4 Evaluation

We evaluate ODINANN to answer the following questions:

- How does ODINANN compare to other updatable ANNS indexes for concurrent inserts and searches? (§4.2)
- How does ODINANN perform in billion-scale datasets compared to other ANNS indexes? (§4.3)
- How do the techniques in ODINANN contribute to the performance of direct insert? (§4.4)
- How do ODINANN’s techniques for inserts impact internal metrics, e.g., disk consumption and index quality? (§4.5)
- How does the number of out-neighbors impact ODINANN’s insert and search performance? (§4.6)
- How does ODINANN perform for workloads with concurrent inserts, deletes, and searches? (§4.7)

4.1 Experimental Setup

Basic configuration. We use one node for evaluation, which has the following configuration:

- **CPU:** 2× 28-core Intel Xeon Gold 6330 @ 2.00GHz;
- **RAM:** 512GB (16× 32GB DDR4 2933MT/s);

- **SSD:** 1× Samsung PM9A3 3.84TB;
- **OS:** Ubuntu 22.04 LTS with Linux kernel 5.15.0.

Datasets. We use three vector datasets for evaluation:

- SIFT100M [12], a dataset containing 100 million `uint8` vectors with a dimension of 128, and 10,000 query vectors.
- DEEP100M [3], a dataset containing 100 million `float` vectors with a dimension of 96, and 10,000 query vectors.
- SIFT1B [12], a dataset containing one billion `uint8` vectors with a dimension of 128, and 10,000 query vectors.

SIFT100M and DEEP100M are subsets of SIFT1B and DEEP1B with 100 million vectors.

Compared systems. We compare ODINANN with two state-of-the-art ANNS indexes: DiskANN [23] and SPFresh [30].

DiskANN is a graph-based index using buffered inserts. ODINANN is implemented based on DiskANN. Unless otherwise stated, we use the following update configurations for them: Insert candidate pool size l_i is 128. Maximum out-neighbors per record R is 96 for SIFT100M and DEEP100M, and 128 for SIFT1B. The maximal number of vectors to prune in delete C is 384. DiskANN merges updates when the in-memory index size reaches 6% of the on-disk index.

For search configurations, they use a beamwidth equal to 4. The default search candidate list l_s is set to 20 for SIFT100M, and 25 for DEEP100M and SIFT1B. The near-root neighbor cache is disabled. For efficiency, we use `io_uring` [13] in replace of `libaio` as their I/O engine.

SPFresh is a variant of SPANN [5], a cluster-based index that divides vectors into clusters. It supports vector updates by updating the target vector in its nearest clusters. To reduce the imbalance of clusters, SPFresh splits and merges them. We configure SPFresh using the same settings as its open-source code. The maximum cluster size is set to 16KB for SIFT and 48KB for DEEP, for the same number of vectors in a cluster.

We do not compare with other on-disk ANNS systems, such as Starling [27], as they are not designed to support updates.

Metrics. ODINANN is designed for online ANNS scenarios, so we evaluate the search performance, search accuracy, and memory usage during concurrent updates. We use multiple search threads to evaluate search performance, as high search throughput is required in real-world workloads (e.g., 5000 QPS [29]). We gradually add search threads to increase throughput while keeping latency relatively low. For ODINANN and DiskANN, we use 32 search threads. For SPFresh, we use 16 search threads in SIFT and 8 threads in DEEP.

4.2 Overall Performance: Insert-Search

In this evaluation, we build indexes using SIFT100M and DEEP100M datasets and then insert another 100 million vectors into them. Multi-thread searches are concurrently executed to show their interference by inserts and merges.

The evaluated systems take different times in the evaluations. This is because we do not insert vectors during

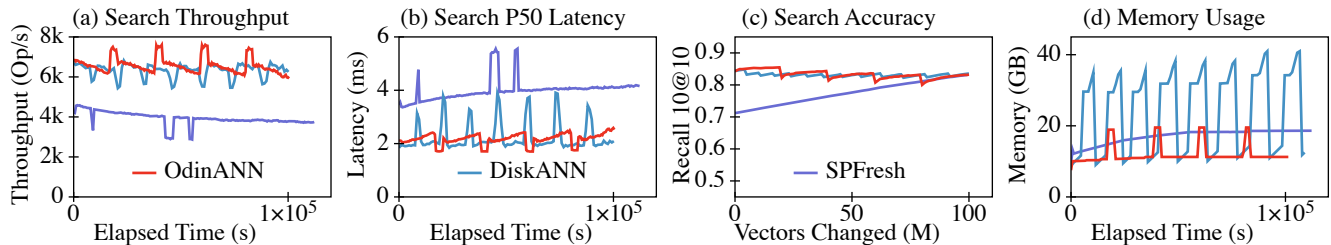


Figure 12: Workload with deletes in SIFT100M dataset, replace the original 100M vectors with another 100M vectors. Note that ODINANN merges at 20% updated vectors, while DiskANN merges at 6% for less memory usage. With similar elapsed time in this figure, ODINANN and SPFresh finish replacing the whole 100M vectors, while DiskANN replaces 50M vectors.

5 Related Work

Billion-Scale ANNS indexes. Real-world workloads do vector search on billions of vectors [7, 22]. Traditional ANNS indexes are stored in memory [4, 7, 18]. To support billion-scale datasets in a single node, they suffer from high prices and limited scalability. To this end, two new methods, *hybrid storage* and *vector compression*, are proposed.

Hybrid storage. Some works rely on hybrid storage media for more capacity, such as SSD and DRAM. DiskANN [24] and Starling [27] are on-disk graph-based indexes. They store the full graph on disk, and compressed vectors in memory for graph navigation. SPANN [5] is an on-disk cluster-based index. It stores the clusters on disk, and a small graph in memory to index the nearest clusters. HM-ANN [21] maintains a hierarchical graph [18] in DRAM and persistent memory.

Some other works seek performance boosts with near-data processing. CXL-ANNS [11] uses processing units near the CXL memory for distance calculation, and SmartANNS [25] uses FPGAs in SmartSSDs for graph traversal. These methods can not only reduce the CPU burden for computation, but mitigate the bandwidth bottleneck by avoiding transferring vectors between CPU and slow-speed storage media.

ODINANN uses commodity SSDs for capacity expansion. It does not rely on near-data processing for better compatibility.

Vector compression. Instead of increasing capacity, some works compress vectors for smaller index sizes in memory. Product quantization (PQ) [8, 14] is a popular vector compression method for ANNS. It splits vectors into equally sized chunks and does clustering for each chunk. Vectors are replaced with per-chunk IDs representing their nearest centroids. This way is used by IVF [2] indexes for less memory footprint. DiskANN [24] also stores PQ-compressed vectors in memory to reduce neighbor reads during graph navigation. LVQ [1] focuses on single-vector compression instead of cross-vector clustering. It uses scalar quantization in each vector to map floating-point vectors to bits, which improves search performance with reduced memory footprint.

ODINANN stores PQ-compressed vectors in memory for

graph navigation. The designs in ODINANN are orthogonal to vector compression methods.

Updatable ANNS indexes. Graph-based DiskANN [23] uses buffered inserts, which interfere with frontend search tasks, motivating our designs for ODINANN. Cluster-based SPFresh [30] supports in-place vector updates. It is more challenging for graph-based ODINANN to support efficient updates than cluster-based SPFresh, due to the intrinsic complexity of graph neighbor maintenance. However, ODINANN benefits from better search accuracy and performance.

Reserving space for efficient updates. ALEX [6] is a learned index that also leverages reserved space (called "gaps" in its paper) for efficient updates. Unlike ALEX, which uses one gap for each new key-value pair to avoid model retraining, ODINANN uses multiple record slots for a single insert, combining record updates in the same page.

6 Conclusion

We argue that direct insert is more suitable for graph-based ANNS indexes and propose ODINANN. It efficiently processes inserts in the on-disk index instead of buffering them. Evaluations show that ODINANN keeps stable insert and search performance in billion-scale vector datasets. This work demonstrates that on-disk graph-based ANNS indexes can benefit from high search accuracy, stable search performance, as well as efficient updates simultaneously using direct insert.

Acknowledgments

We sincerely thank our shepherd, Bingzhe Li, and the anonymous reviewers for their valuable feedback. This work is supported by the National Key R&D Program of China (Grant No. 2024YFE0203300), the National Natural Science Foundation of China (Grant No. 62332011), and Beijing Natural Science Foundation (Grant No. L242016).